

Paper阅读：Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First

目录

- 一：背景
- 二：引言
- 三：case study
 - 研究Agent SQL 查询的负载特征
- 四 Agent-First Data System Architecture
 - 查询方式的重构
 - 查询优化的重构
 - 事务管理的重构
- 总结

一：背景

UC Berkeley 于2025 年 8 月在 arXiv 上发表论文《Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First》。该论文主要分享讨论了 **AI Agent时代需要一个什么样的数据库？**

该论文发表后立即在 Hacker News 等技术社区引发了热烈讨论。这表明它成功地抓住了当前 AI Agents 时代的核心痛点。同时论文作者团队包括了数据系统和分布式计算领域的领军人物（Matei Zaharia 是 Apache Spark 的创始人，Ion Stoica Databricks 联合创始人兼执行主席）



Supporting Our AI Overlords- Redesigning Data Systems to be Agent-First.pdf

854.1KB

二：引言

LLM（大语言模型）的推理能力会越来越强，对数据的请求会更频繁；这些请求中许多并不是一次性找到准确的solution，而是**agentic speculation（agent推测）**——高吞吐量的探索性

查询，然后验证。来确定最佳solution。

比如：为延误的航班识别新的机组人员的task。agent 需要提出各种假设事务，然后获取到执行完各种假设事务后的结果集返回给用户；而对于数据库来说会有大量的回滚操作。

论文提出满足agentic speculation数据库的特点：

1. 高吞吐量
2. 异构的、跨越粗粒度数据和元数据探索、部分和完整的解决方案制定和验证——允许 agent 早期阶段以近似或不完整的输出取得进展
3. 有冗余性：许多请求可能访问相似的数据或执行重叠的操作，提供共享计算或消除冗余工作的机会
4. 可引导的：由于推测本质上是探索性的，应该超越传统的 query-answer模式，并允许数据系统更直接地与 LLM 代理通信。

三：case study

研究Agent SQL 查询的负载特征

为了弄清楚 Agent 使用 SQL 有哪些特征，论文基于 BIRD Text2SQL benchmark 进行分析，使用了 GPT-4o-mini 和 Qwen2.5-Coder-7B-Instruct 两个模型进行对比，后端数据库为duckdb

结论 1：单 Agent 多轮尝试或多 Agent 并发尝试有助于提升 SQL 查询正确率

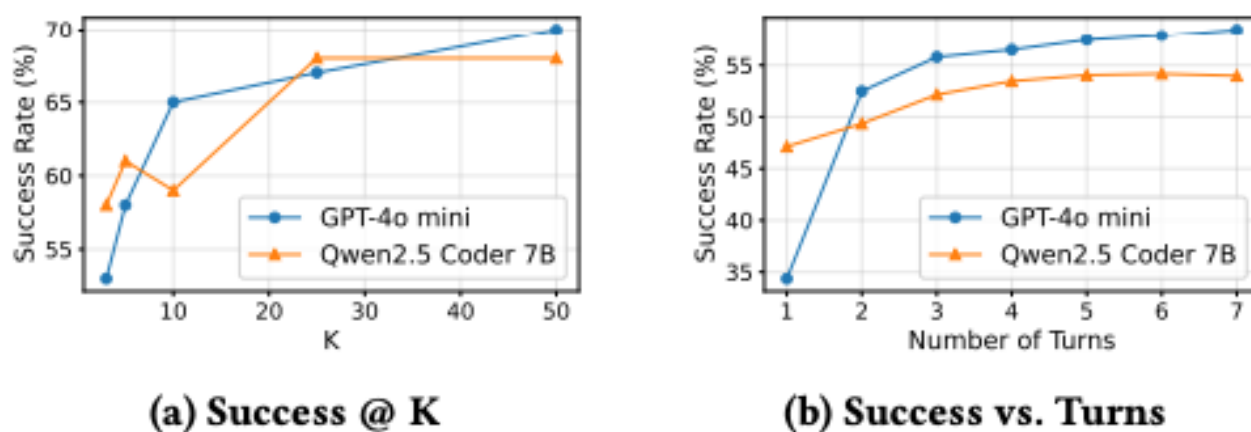
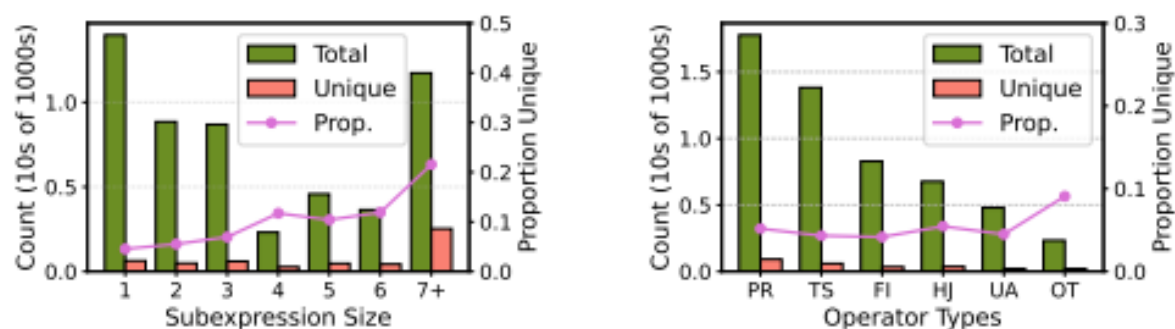


Figure 1: Results on the BIRD dataset

结论 2：Agent 有大量重复的 SQL 查询，占总查询数的 80%~90%

在查询中，每个大小的不同子计划的数量通常只是总数的一小部分，不到 10-20%，代表了相当大的共享计算潜力。



(a) versus subexpression size.

(b) versus root operation.

Figure 2: Total vs. unique subexpressions (count and proportion) across 50 attempts generated by GPT-4o-mini per problem, aggregated over the full BIRD dataset. Here, PR=Projection, TS=Scan, FI=Filter, HJ=Hash Join, UA=Aggregate, OT=other operations.

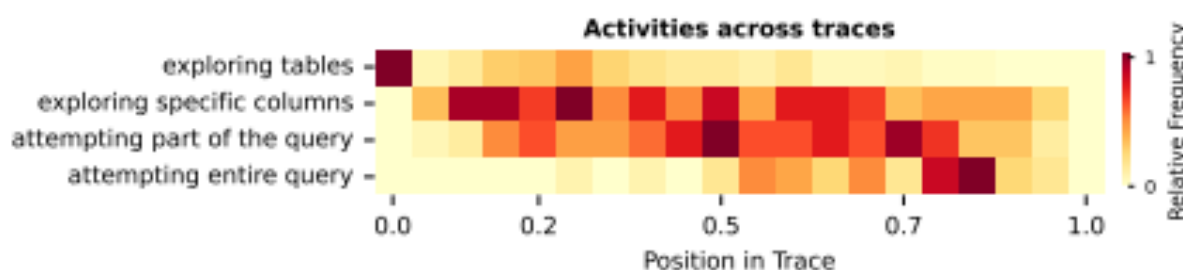
结论 3: Agent 的 SQL 查询类型是多样的

给 Agent 一个数据查询的任务，它并不会立刻生成结果查询 SQL，而是会先进行一些探索，比如找到有哪些表、表中有哪些列等。

论文把 Agent 完成一个数据查询任务所执行的 SQL 划分成 4 个类型：

1. **探索表** (exploring tables)：查询数据库中都有哪些表。
2. **探索列** (exploring specific columns)：查询指定表中有哪些列。
3. **中间查询** (attempting part of the query)：基于前 2 步的信息，构建中间查询，开始尝试查询相关数据。
4. **完整查询** (attempting entire query)：根据子查询生成完整的 SQL，并执行查询，得到最终结果。

可以看到，Agent 会先进行粗粒度的元数据查询，然后再是进行从部分到整体的数据查询，直到完成任务。



结论 4: 适当的引导可以有效降低 Agent 完成一项任务的轮次

如果 Agent 在大部分时间里都在试错，就意味着数据库大部分时间里都是在做无用功，浪费资源。

如果给 Agent 适当的引导，比如告诉 Agent 完成这项任务所需的表和列有哪些，能否减少这些无用功呢？

论文对此做了对比实验，结果数据如下：

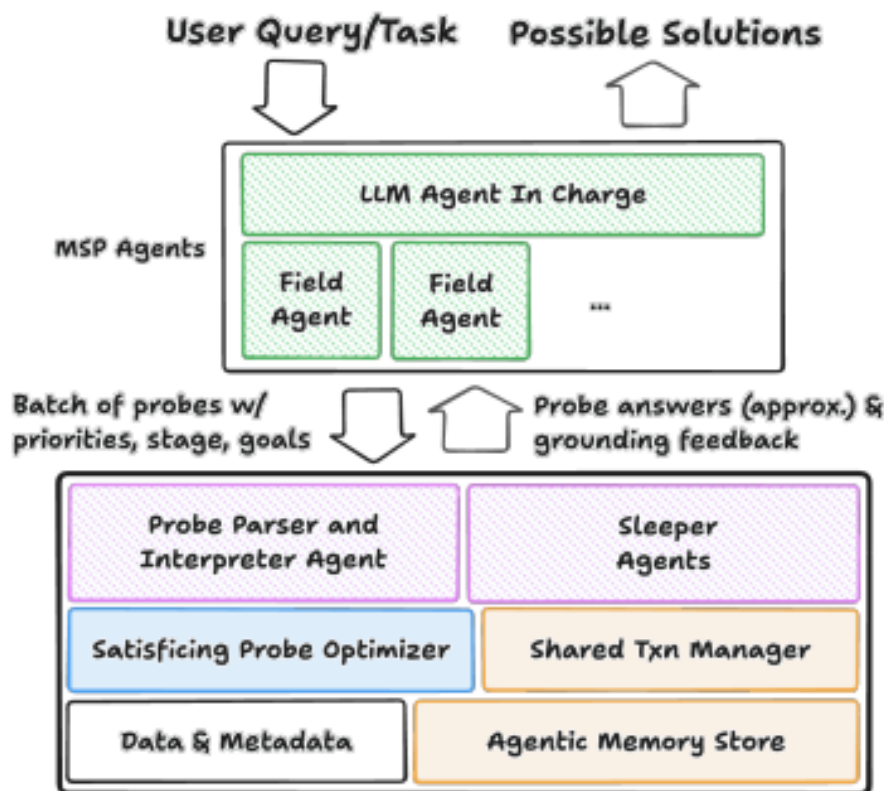
Table 1: Mean activity counts per agent trace, averaged across all traces, with and without human expert-provided hints.

Activity	Avg (No Hints)	Avg (w/ Hints)	Reduction (%)
exploring tables	3.44	2.95	-14.2
exploring specific columns	3.56	2.57	-27.7
attempting part of the query	4.28	2.71	-36.6
attempting entire query	1.26	1.05	-16.6
all SQL queries	12.67	10.38	-18.1

显然，为人类设计的传统数据库已无法高效支撑 Agentic Speculation，它需要从一个被动的查询执行者，转变为一个能够与 Agent 主动协作的伙伴，也即 Agent-First 数据库。

四 Agent-First Data System Architecture

论文提出了 Agent-First 数据库的架构畅想，如下图所示：



Probe Parser and interpreter agent: 代替传统的sql解析器，主要功能解析probe，随后提交给后端执行

Sleeper agents: 按需唤醒多个sleeper agents, 主要提供辅助信息&&成本建议&&下一步查询建议等

Shared txn Manager: 处理潜在事务中涉及到的状态冗余

agentic memory store: 存储所获得到的基础信息，统计信息等

查询方式的重构

Agent 与数据库交互的语言，不再是 SQL，而是一批 Probe。 Probe 不仅包含了 SQL，还有附带一段由自然语言描述的背景信息，这主要起到引导作用，有助于减少反复试错。

背景信息可以包含如下内容：

- **查询阶段：**元数据探索阶段和数据查询阶段对数据的精度、调度优先级都不一样。
- **查询精度。**比如可以指定查询结果不需要 100% 精确，80% 就好
- **终止条件。**比如，在全表扫描请求中，指定查询终止条件为扫描 80% 数据即可，或查询时长不超过 1ms

查询优化的重构

查询优化器的目标不再是传统数据库那样的“最快地给单次查询一个精确的结果”，而是“**用最小的总时间代价，完成本批次以及未来批次的 Probes 查询任务**”。查询结果不一定是精确的，只要能够完成 MSP Agents 交代的任务即可。

对于当前批次优化（**Intra-Probe Optimization**）：

- **近似替代精确。**比如，优化器发现当前处于探索阶段，可以适当降低查询精度，减少资源消耗。
- **避免“因小便宜吃大亏”。**比如，当前查询如果返回一个低精度的结果，很可能引发更多轮的查询，得不偿失。这就需要优化器综合考虑当前数据库自身的状态、系统资源，决定在本轮或下一轮执行精确查询。
- **动态调整计划。**执行计划从静态转变为动态，比如，对于 A 列，最初优化的结果是设置查询精度为 80%，但随着查询的执行，发现系统资源不足，就主动将精度调整为 60%。

对于多轮批次优化（**Inter-Probe Optimizations**）：

- **基于增量信息做剪枝，避免做无用功**
- **提前准备好下一批查询的结果：**对于近期重复查询的结果进行缓存等手段

事务管理的重构

Agent 会存在大量的 “What-if” 式的探索流程。论文里提到，Neon 数据库统计数据显示，**Agent 创建的数据分支数量是人类创建的 20 倍、Agent 执行数据回滚的次数是人类的 50 倍。**

传统的MVCC机制已无法应对这种场景。MVCC 的目的让多个并发事务感觉不到对方的存在，互不干扰。它处理的是短暂的、并行的“**现在**”。它的所有版本最终都会被清理，历史永远是一条直线，是**小而短暂的**。

而 Agent 的 “What-if” 是要创建无数个平行的“**未来**”，它需要同时保留成百上千个数据快照，在每个快照上运行调试，比较结果，然后选择最优的那个“**未来**”。所以，它的历史是一个庞大的树状结构，是**大而长寿的**。

因此我们需要新的并发机制，以实现大规模并行分叉。这类似于增强版的 MVCC：分叉可能数千个几乎相同的快照，然后回滚除一个之外的所有快照。与很少进行回滚的传统数据系统不同，我们需要超快速的回滚（即快速中止失败的分支）。

总结

总结 Agent SQL 查询的负载特征以及数据库所需的能力如下：

1	Agent SQL 查询负载特征	Agent SQL查询场景分析	Agent-First 数
2	高吞吐量	agent发起的请求并不能一次性找到准确的 solution，会发起多次探索性查询	批量 Probe 处
3	高重复率	Agent 有大量重复的 SQL 查询，占总查询数的 80%~90%	查询优化器主
4	多样性	Agent 完成一个数据查询任务所执行的 SQL 划分成 4 个类型： 1. 探索表 2. 探索列 3. 中间查询 4. 完整查询	Agent 记忆存
5	可引导	Agent 在大部分时间里都在试错，适当的引导可以有效降低 Agent 完成一项任务的轮次	Probe 替代 S
6	大量 “What-if” 探索	agent发起大量的尝试性事务，并基于此做出最优选择	数据分支替代

